

#There are 3 questions we need to answer today
#data link:https://docs.google.com/spreadsheets/d/16u64opSCVr-W-i8ak33C16DM8eU7rgeo/edit?gid=1731826593#gid=17318265
#1. Are we seeing any lead quality trends over time (improving, declining)? Are they statistically significant?
#2. What can we learn about the drivers of "lead quality" from this dataset? What segments - where the ad was shown,
#3. If the advertiser says they will increase our CPL by 20% (i.e., \$30 to \$33) if we increase our lead quality by 2

#Part 1 Data Exploration

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier, IsolationForest
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score, roc_curve, precision_recall_curve, average_precision_score
from scipy.stats import multivariate_normal
from tabulate import tabulate
import io
import warnings
warnings.filterwarnings("ignore")
```

```
lead_op = pd.read_excel("/content/Analyst_case_study_dataset_1(1).xls")
lead_op.head()
```

	LeadCreated	FirstName	Email	VendorLeadID	CallStatus	WidgetName	PublisherZoneName
0	2009-07-01 01:24:29.563	Dorinda	kanani@sandwichisles.net	FDF81FDA-A649-437B-B99C-FDDE74F7971B	NaN	w-302252-DebtReduction1-1DC-CreditSolutions	TopLeft-302252
1	2009-04-13 15:27:35.990	Presetta	clerk2@ustconline.net	4190ACB7-5026-416C-B987-ED8AD427D5E6	NaN	w-300250-DebtReduction1-1DC-Head2	TopLeft-302252
2	2009-04-21 07:22:59.000	Gina	wagoner_gina@yahoo.com	hFg80jf_R0CRN55hdhWILw	Unable to contact - Bad Contact Information	w-300250-DebtReduction1-1DC-Head2	TopLeft-302252
3	2009-08-03 19:39:02.000	Kari	usa4ley@yahoo.com	jB01QgYZxkWArI9jWxuurfw	Contacted - Doesn't Qualify	w-302252-DebtReduction1-1DC-white	TopLeft-302252
4	2009-04-13 10:18:37.490	Stephanie	sr1lambert@embarqmail.com	D5B32074-458E-40EC-B185-1FEF20AC626D	NaN	w-300250-DebtReduction1-1DC-BlueMeter	TopLeft-302252

5 rows x 24 columns

```
lead_op.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3021 entries, 0 to 3020
Data columns (total 24 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   LeadCreated                          3021 non-null  datetime64[ns]
1   FirstName                            3021 non-null  object
2   Email                                3021 non-null  object
3   VendorLeadID                        3013 non-null  object
4   CallStatus                          881 non-null   object
5   WidgetName                          3021 non-null  object
6   PublisherZoneName                   3021 non-null  object
7   PublisherCampaignName               3021 non-null  object
8   AddressScore                        1171 non-null  float64
9   PhoneScore                          1393 non-null  float64
10  AdvertiserCampaignName              3021 non-null  object
11  State                               3021 non-null  object
12  DebtLevel                           3021 non-null  object
13  IP Address                          0 non-null     float64
14  Partner                             3021 non-null  object
15  ReferralDomain                      2506 non-null  object
16  MarketingCampaign                   2749 non-null  object
17  AdGroup                             2749 non-null  object
18  Keyword                             979 non-null   object
```



```
19 SearchQuery          1265 non-null object
20 ReferralURL          2506 non-null object
21 ReferralURL Parameters 2283 non-null object
22 LandingPageURL        3021 non-null object
23 Landing Page URL Parameters 3021 non-null object
dtypes: datetime64[ns](1), float64(3), object(20)
memory usage: 566.6+ KB
```

lead_op.describe()

	LeadCreated	AddressScore	PhoneScore	IP Address	
count	3021	1171.000000	1393.000000	0.0	
mean	2009-06-25 01:25:50.416092416	4.286080	3.903087	NaN	
min	2009-04-02 18:52:41	1.000000	1.000000	NaN	
25%	2009-05-13 22:56:40.676999936	4.000000	3.000000	NaN	
50%	2009-06-18 19:45:45	5.000000	4.000000	NaN	
75%	2009-08-03 12:21:34.972999936	5.000000	5.000000	NaN	
max	2009-09-30 21:00:20.080000	5.000000	5.000000	NaN	
std	NaN	1.345192	1.101826	NaN	

lead_op["CallStatus"].unique()

```
array([nan, 'Unable to contact - Bad Contact Information',
        'Contacted - Doesn't Qualify', 'Closed', 'EP Received',
        'EP Confirmed', 'Contacted - Invalid Profile', 'EP Sent'],
      dtype=object)
```

lead_op["CallStatus"].value_counts()

	count
CallStatus	
Closed	245
Contacted - Doesn't Qualify	212
Unable to contact - Bad Contact Information	191
EP Confirmed	130
Contacted - Invalid Profile	85
EP Sent	16
EP Received	2

dtype: int64

lead_op["Partner"].value_counts()

	count
Partner	
google	979
yahoo	958
Google	639
Call_Center	271
AdKnowledge	171
Advertise.com	3

dtype: int64

lead_op["MarketingCampaign"].value_counts()



	count
MarketingCampaign	
DebtReductionInc	1473
Debt Holding Tank	338
Debt General	329
Financial Services	171
Debt Volume	143
Credit	100
state	97
Debt Consolidation	34
DebtReductionInc – YSM Restructure (05/29/09)	23
Debt Volume_CA	12
Debt Volume_TX	7
Debt Volume_MI	5
Debt_General_-_SN	5
state – YSM Restructure (05/29/09)	3
Debt	3
Debt_Consolidation_-_SN	2
Debt_Holding_Tank_-_SN	1
Debt_Volume_-_SN	1
Debt Reduction	1
Credit_-_SN	1

dtype: int64

lead_op["SearchQuery"].value_counts()

	count
SearchQuery	
debt consolidation	27
www.chase.com creditcards payment	25
debt cures	18
consumer debt initiative	11
debt rescue usa	10
...	...
http://www.getridofthings.com	1
debt Recovery	1
debt consolidation even with bad credit	1
debt collectors	1
collection laws	1

961 rows x 1 columns

dtype: int64

lead_op.isnull().sum()



	0
LeadCreated	0
FirstName	0
Email	0
VendorLeadID	8
CallStatus	2140
WidgetName	0
PublisherZoneName	0
PublisherCampaignName	0
AddressScore	1850
PhoneScore	1628
AdvertiserCampaignName	0
State	0
DebtLevel	0
IP Address	3021
Partner	0
ReferralDomain	515
MarketingCampaign	272
AdGroup	272
Keyword	2042
SearchQuery	1756
ReferralURL	515
ReferralURL Parameters	738
LandingPageURL	0
Landing Page URL Parameters	0

dtype: int64

#Part2:Are we seeing any lead quality trends over time (improving, declining)? decline
#Are they statistically significant?yes

```
time_min = lead_op['LeadCreated'].min()
time_max = lead_op['LeadCreated'].max()
time_span = time_max - time_min

print(time_min)
print(time_max)
print("span", time_span)
```

```
2009-04-02 18:52:41
2009-09-30 21:00:20.080000
span 181 days 02:07:39.080000
```

```
lead_op['LeadCreated'] = pd.to_datetime(lead_op['LeadCreated'], errors='coerce')
lead_op['day'] = lead_op['LeadCreated'].dt.date
daily_counts = lead_op.groupby('day').size().reset_index(name='lead_count')
daily_counts = daily_counts.sort_values('day')
print(daily_counts)
```

	day	lead_count
0	2009-04-02	2
1	2009-04-03	17
2	2009-04-04	11
3	2009-04-05	7
4	2009-04-06	12
..	...	...
177	2009-09-26	12
178	2009-09-27	16
179	2009-09-28	15
180	2009-09-29	15



```
181 2009-09-30      11
```

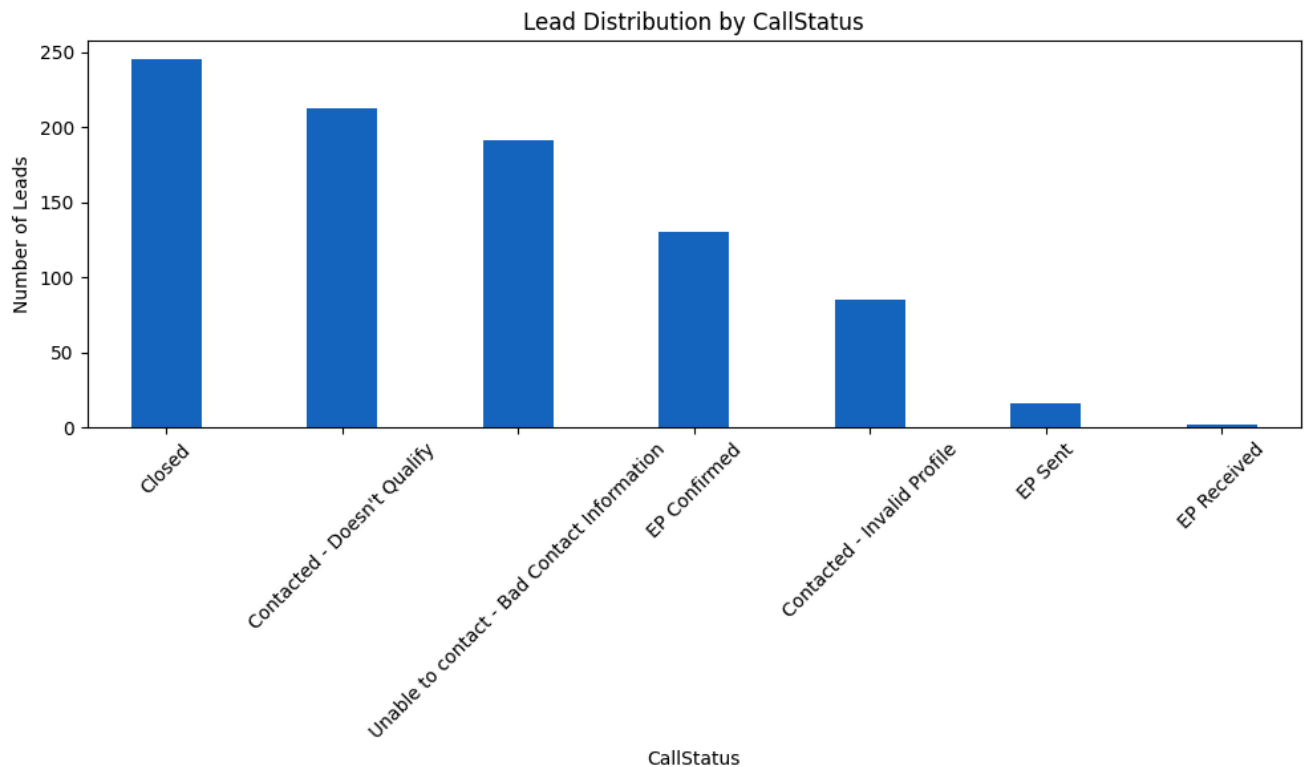
```
[182 rows x 2 columns]
```

```
lead_op['week'] = lead_op['LeadCreated'].dt.to_period('W')
weekly_counts = lead_op.groupby('week').size().reset_index(name='lead_count')
weekly_counts = weekly_counts.sort_values('week')
print(weekly_counts)
```

	week	lead_count
0	2009-03-30/2009-04-05	37
1	2009-04-06/2009-04-12	119
2	2009-04-13/2009-04-19	125
3	2009-04-20/2009-04-26	155
4	2009-04-27/2009-05-03	129
5	2009-05-04/2009-05-10	113
6	2009-05-11/2009-05-17	166
7	2009-05-18/2009-05-24	122
8	2009-05-25/2009-05-31	132
9	2009-06-01/2009-06-07	137
10	2009-06-08/2009-06-14	165
11	2009-06-15/2009-06-21	153
12	2009-06-22/2009-06-28	107
13	2009-06-29/2009-07-05	101
14	2009-07-06/2009-07-12	104
15	2009-07-13/2009-07-19	115
16	2009-07-20/2009-07-26	144
17	2009-07-27/2009-08-02	136
18	2009-08-03/2009-08-09	95
19	2009-08-10/2009-08-16	116
20	2009-08-17/2009-08-23	104
21	2009-08-24/2009-08-30	89
22	2009-08-31/2009-09-06	74
23	2009-09-07/2009-09-13	65
24	2009-09-14/2009-09-20	95
25	2009-09-21/2009-09-27	82
26	2009-09-28/2009-10-04	41

```
status_counts = lead_op['CallStatus'].value_counts()
plt.figure(figsize=(10,6))
status_counts.plot(kind='bar', color='#1565C0', width=0.4)
plt.ylabel('Number of Leads')
plt.xlabel('CallStatus')
plt.title('Lead Distribution by CallStatus')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

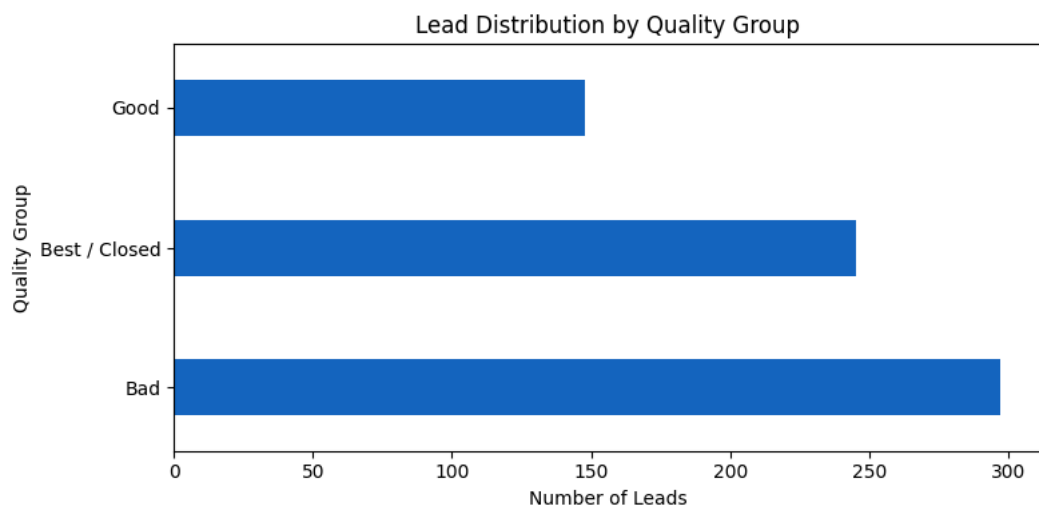




```
quality_group_map = {
    'Closed': 'Best / Closed',
    'EP Sent': 'Good',
    'EP Received': 'Good',
    'EP Confirmed': 'Good',
    'Unable to Contact': 'Bad',
    'Contacted - Invalid Profile': 'Bad',
    'Contacted - Doesn't Qualify': 'Bad',
    'Unknown': 'Unknown / Neutral'
}
lead_op['QualityGroup'] = lead_op['CallStatus'].map(quality_group_map)

quality_counts = lead_op['QualityGroup'].value_counts()

plt.figure(figsize=(8,4))
quality_counts.plot(kind='barh', color='#1565C0', width=0.4)
plt.xlabel('Number of Leads')
plt.ylabel('Quality Group')
plt.title('Lead Distribution by Quality Group')
plt.tight_layout()
plt.show()
```



```

quality_score_map = {
    'Closed': 1,
    'EP Sent': 0.75,
    'EP Received': 0.75,
    'EP Confirmed': 0.75,
    'Unable to Contact': 0,
    'Contacted - Invalid Profile': 0,
    'Contacted - Doesn\'t Qualify': 0,
    'Unknown': 0.5
}
lead_op['lead_quality_score'] = lead_op['CallStatus'].map(quality_score_map)
lead_op['LeadCreated'] = pd.to_datetime(lead_op['LeadCreated'], errors='coerce')
lead_op['week'] = lead_op['LeadCreated'].dt.to_period('W')

weekly_quality = lead_op.groupby('week').agg(
    avg_quality=('lead_quality_score', 'mean'),
    total_leads=('VendorLeadID', 'count')
).reset_index()
print(weekly_quality)

```

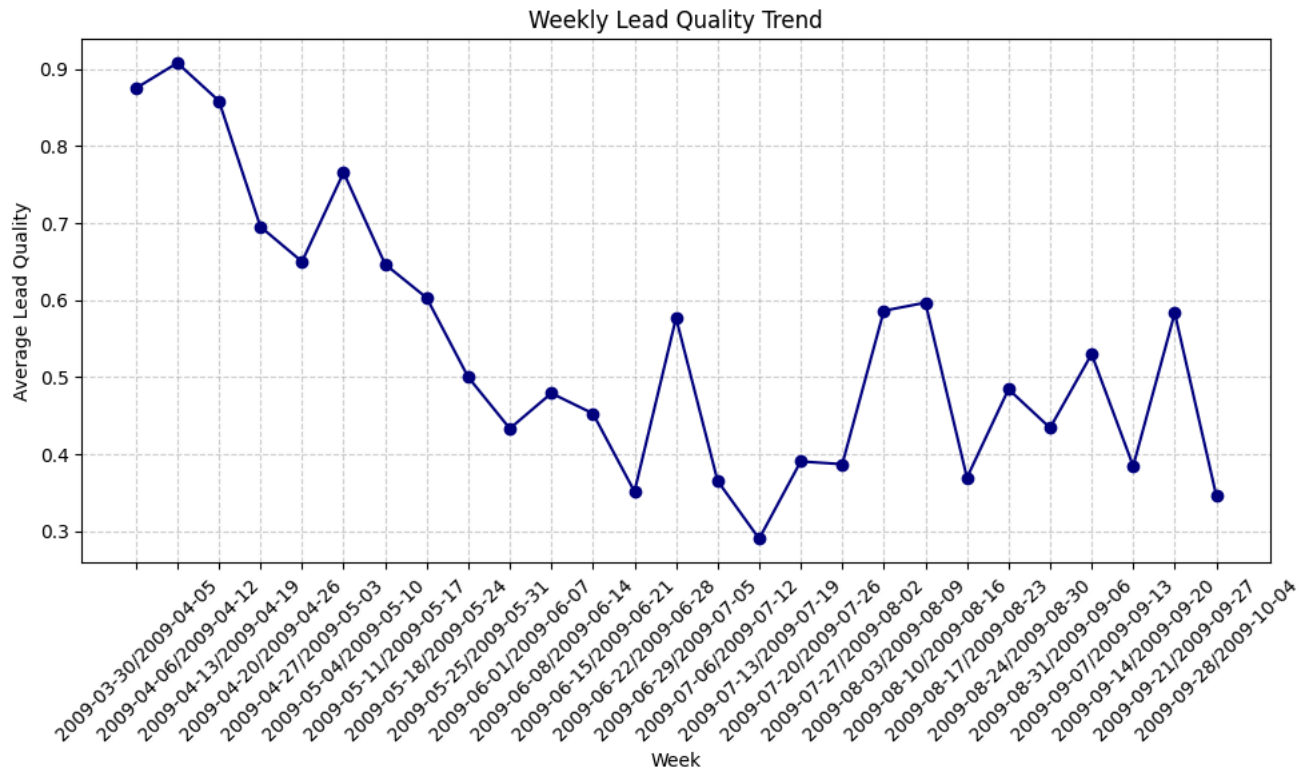
	week	avg_quality	total_leads
0	2009-03-30/2009-04-05	0.875000	37
1	2009-04-06/2009-04-12	0.907895	119
2	2009-04-13/2009-04-19	0.858696	125
3	2009-04-20/2009-04-26	0.695312	154
4	2009-04-27/2009-05-03	0.650000	129
5	2009-05-04/2009-05-10	0.765625	113
6	2009-05-11/2009-05-17	0.646552	165
7	2009-05-18/2009-05-24	0.602941	122
8	2009-05-25/2009-05-31	0.500000	132
9	2009-06-01/2009-06-07	0.433333	137
10	2009-06-08/2009-06-14	0.479167	164
11	2009-06-15/2009-06-21	0.452830	151
12	2009-06-22/2009-06-28	0.351562	107
13	2009-06-29/2009-07-05	0.576923	100
14	2009-07-06/2009-07-12	0.365385	104
15	2009-07-13/2009-07-19	0.290323	115
16	2009-07-20/2009-07-26	0.390625	144
17	2009-07-27/2009-08-02	0.387097	136
18	2009-08-03/2009-08-09	0.586207	95
19	2009-08-10/2009-08-16	0.596774	116
20	2009-08-17/2009-08-23	0.369565	103
21	2009-08-24/2009-08-30	0.484375	88
22	2009-08-31/2009-09-06	0.434211	74
23	2009-09-07/2009-09-13	0.530000	65
24	2009-09-14/2009-09-20	0.384615	95
25	2009-09-21/2009-09-27	0.583333	82
26	2009-09-28/2009-10-04	0.346154	41

```

plt.figure(figsize=(10,6))
plt.plot(weekly_quality['week'].astype(str), weekly_quality['avg_quality'], marker='o', color='navy')
plt.xticks(rotation=45)
plt.ylabel('Average Lead Quality')
plt.xlabel('Week')
plt.title('Weekly Lead Quality Trend')
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()

```





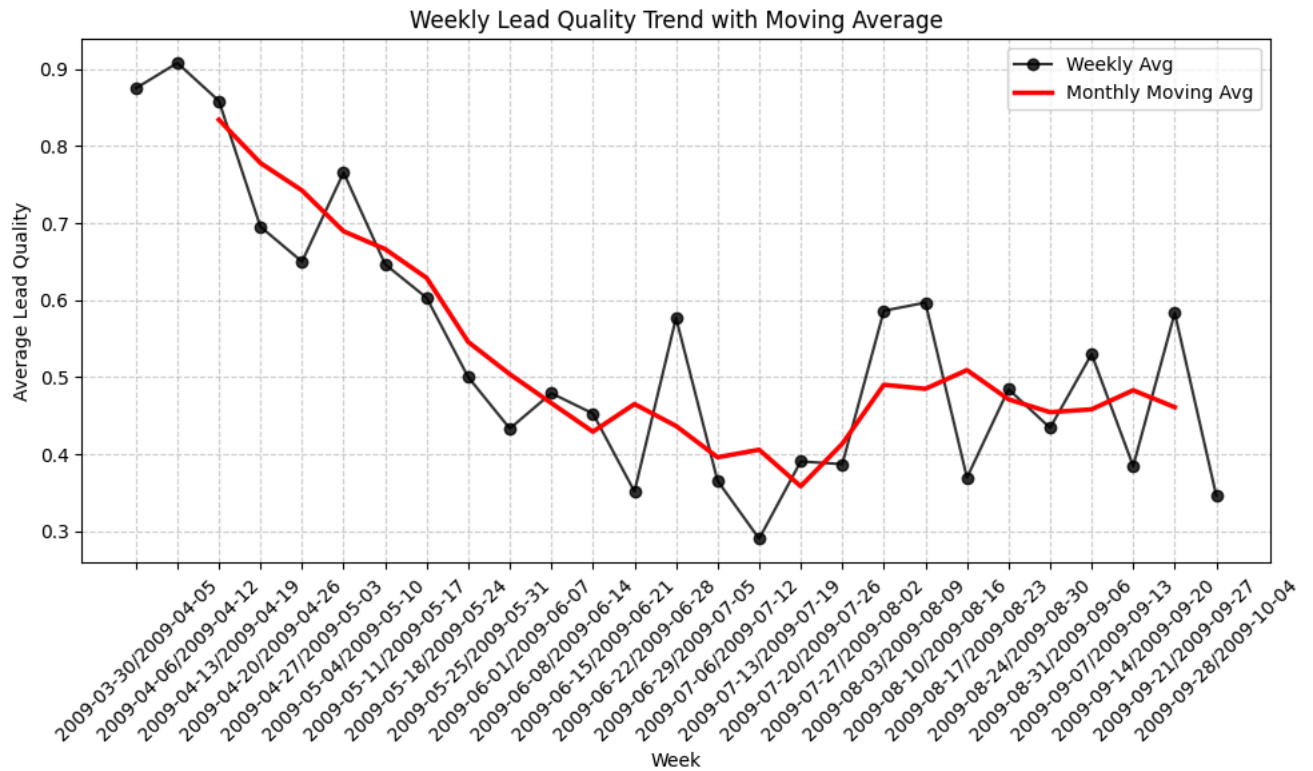
```
from statsmodels.tsa.stattools import adfuller
result = adfuller(weekly_quality['avg_quality'])
print('ADF Statistic:', result[0])
print('p-value:', result[1])
```

```
ADF Statistic: -2.491600276593572
p-value: 0.11753146788830476
```

```
s_diff = weekly_quality['avg_quality'].diff().dropna()
```

```
weekly_quality['avg_quality_ma'] = weekly_quality['avg_quality'].rolling(window=4, center=True).mean()
plt.figure(figsize=(10,6))
plt.plot(weekly_quality['week'].astype(str), weekly_quality['avg_quality'], marker='o', color='black', alpha=0.8, label='Avg Quality')
plt.plot(weekly_quality['week'].astype(str), weekly_quality['avg_quality_ma'], color='red', linewidth=2.5, label='Moving Average')
plt.xticks(rotation=45)
plt.ylabel('Average Lead Quality')
plt.xlabel('Week')
plt.title('Weekly Lead Quality Trend with Moving Average')
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.tight_layout()
plt.show()
```





```
import statsmodels.api as sm
x = np.arange(len(weekly_quality))
y = weekly_quality['avg_quality']
X = sm.add_constant(x)
model = sm.OLS(y, X).fit()
plt.figure(figsize=(10,6))
plt.plot(x, y, marker='o', alpha=0.8, color='black', label='Weekly Avg')
plt.plot(x, model.predict(X), color='red', linewidth=2.5, label='OLS Trend')

pred = model.get_prediction(X)
pred_ci = pred.conf_int()
plt.fill_between(x, pred_ci[:,0], pred_ci[:,1], color='red', alpha=0.2)

plt.xticks(rotation=45)
plt.ylabel('Average Lead Quality')
plt.xlabel('Week')
plt.title('Weekly Lead Quality Trend with Linear Regression')
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.tight_layout()
plt.show()
```



```
import statsmodels.api as sm
X = sm.add_constant(np.arange(len(weekly_quality)))
y = weekly_quality['avg_quality']

model = sm.OLS(y, X).fit()
print(model.summary())
```

OLS Regression Results

```
=====
Dep. Variable:    avg_quality    R-squared:                0.466
Model:            OLS           Adj. R-squared:            0.444
Method:            Least Squares   F-statistic:                21.80
Date:              Sun, 22 Feb 2026   Prob (F-statistic):        8.80e-05
Time:              02:01:14         Log-Likelihood:            18.287
No. Observations:    27           AIC:                      -32.57
Df Residuals:        25           BIC:                      -29.98
Df Model:            1
Covariance Type:    nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	0.7302	0.048	15.267	0.000	0.632	0.829
x1	-0.0147	0.003	-4.669	0.000	-0.021	-0.008

```
=====
Omnibus:                2.548    Durbin-Watson:            1.040
Prob(Omnibus):           0.280    Jarque-Bera (JB):         1.240
Skew:                    0.015    Prob(JB):                 0.538
Kurtosis:                1.951    Cond. No.:                29.6
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

#Conclusion: The weekly lead quality shows a declining trend over time. The trend is statistically significant, with

#Part3:What can we learn about the drivers of "lead quality" from this dataset?

#What segments – where the ad was shown, what kind of person filled out the ad, what kind of ad did they see – have

```
cols_to_drop = [
    'FirstName',
    'VendorLeadID',
    'IP Address',
    'CallStatus',
    'Email',
```



```
'ReferralURL Parameters',
'Landing Page URL Parameters',
'LeadCreated',
'day',
'week',

]
lead_op_clean = lead_op.drop(columns=cols_to_drop)
print("rest_col", lead_op_clean.columns.tolist())
lead_op_clean.to_csv("data_clean.csv", index=False)
```

```
rest_col ['WidgetName', 'PublisherZoneName', 'PublisherCampaignName', 'AddressScore', 'PhoneScore', 'AdvertiserCampai
```

```
lead_op_clean['QualityGroup'].value_counts()
```

	count
QualityGroup	
Bad	297
Best / Closed	245
Good	148

```
dtype: int64
```

```
mapping = {
    'Best / Closed': 3,
    'Good': 2,
    'Bad': 1,
}

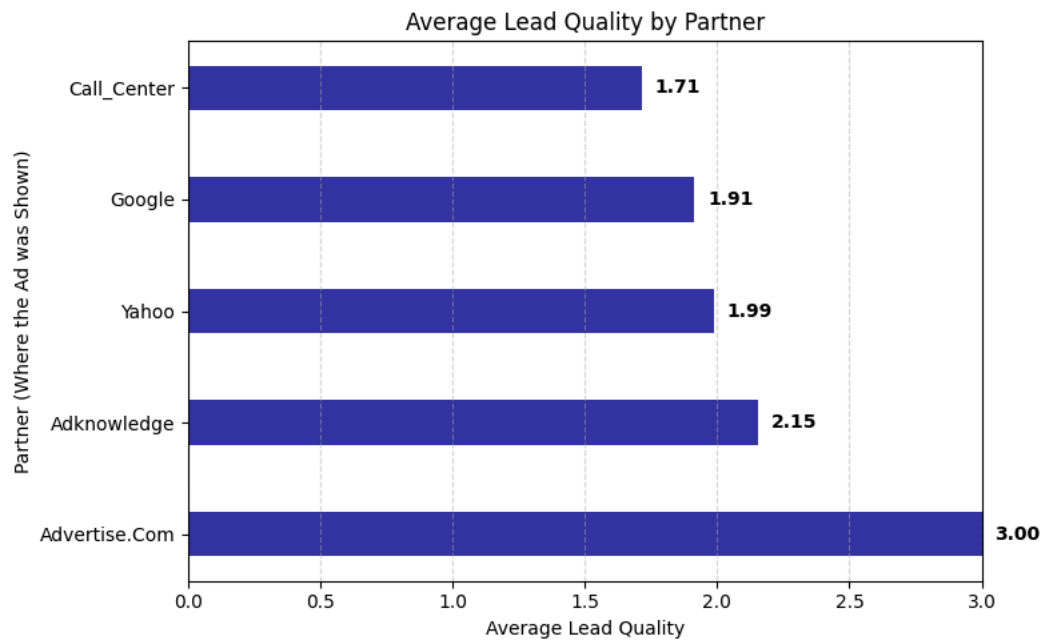
lead_op_clean['QualityNumeric'] = (
    lead_op_clean['QualityGroup']
    .str.strip()
    .map(mapping)
)
lead_op_clean['Partner'] = lead_op_clean['Partner'].str.strip().str.title()
partner_summary = lead_op_clean.groupby('Partner')['QualityNumeric'].mean().sort_values(ascending=False)
print(partner_summary)
```

Partner	
Advertise.Com	3.000000
Adknowledge	2.152174
Yahoo	1.985915
Google	1.912651
Call_Center	1.714286

Name: QualityNumeric, dtype: float64

```
#where the ad has shown?
plt.figure(figsize=(8,5))
plt.barh(partner_summary.index, partner_summary.values, color='darkblue', alpha=0.8, height=0.4)
for i, v in enumerate(partner_summary.values):
    plt.text(v + 0.05, i, f"{v:.2f}", va='center', fontweight='bold')
plt.xlabel('Average Lead Quality')
plt.ylabel('Partner (Where the Ad was Shown)')
plt.title('Average Lead Quality by Partner')
plt.xlim(0, 3)
plt.grid(axis='x', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```





```
#what kind of person filled out the ad?
lead_op_clean.groupby('State').agg(
    avg_quality=('QualityNumeric', 'mean'),
    lead_count=('QualityNumeric', 'count')
).sort_values('avg_quality', ascending=False)
```



	avg_quality	lead_count	
State			
AK	3.000000	1	
IA	2.500000	2	
AR	2.333333	6	
OK	2.222222	18	
NM	2.166667	6	
PA	2.166667	12	
HI	2.090909	11	
AL	2.076923	26	
VA	2.064516	31	
TX	2.044776	67	
CA	2.044643	112	
MT	2.000000	6	
NV	2.000000	13	
NE	2.000000	6	
CT	2.000000	7	
IL	2.000000	32	
WV	2.000000	13	
CO	1.904762	21	
WA	1.857143	7	
LA	1.812500	16	
FL	1.809524	63	
MA	1.800000	20	
MI	1.800000	35	
NY	1.775862	58	
MO	1.750000	20	
MD	1.736842	19	
AZ	1.718750	32	
OR	1.647059	17	
IN	1.428571	7	
SD	1.333333	3	
DC	1.000000	1	
ND	1.000000	2	

```

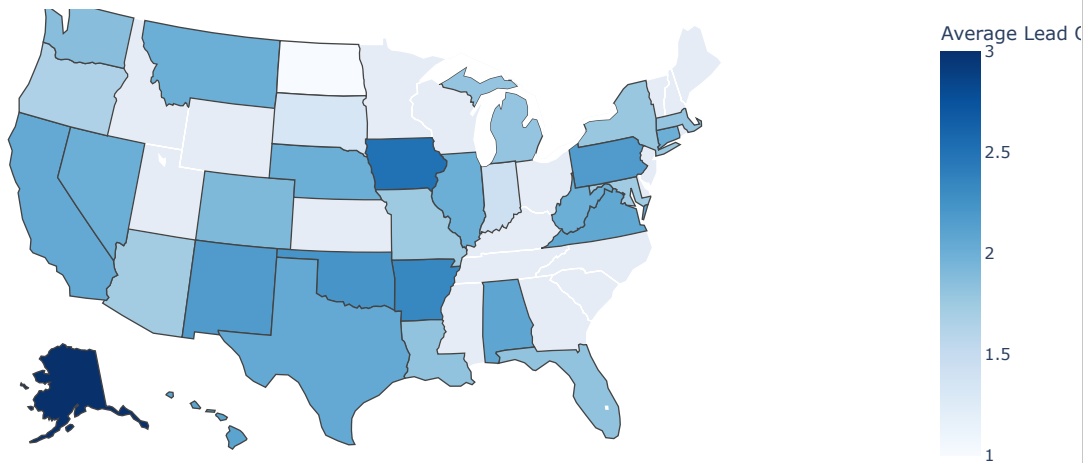
import pandas as pd
import plotly.express as px
state_summary = lead_op_clean.groupby('State').agg(
    avg_quality=('QualityNumeric', 'mean'),
    lead_count=('QualityNumeric', 'count')
).reset_index()
fig = px.choropleth(
    state_summary,
    locations='State',
    locationmode='USA-states',
    color='avg_quality',
    color_continuous_scale='blues',
    scope='usa',
    hover_data={'lead_count':True, 'avg_quality':':.2f'},
    labels={'avg_quality':'Average Lead Quality'},
    title='Average Lead Quality by State'
)

fig.show()

```

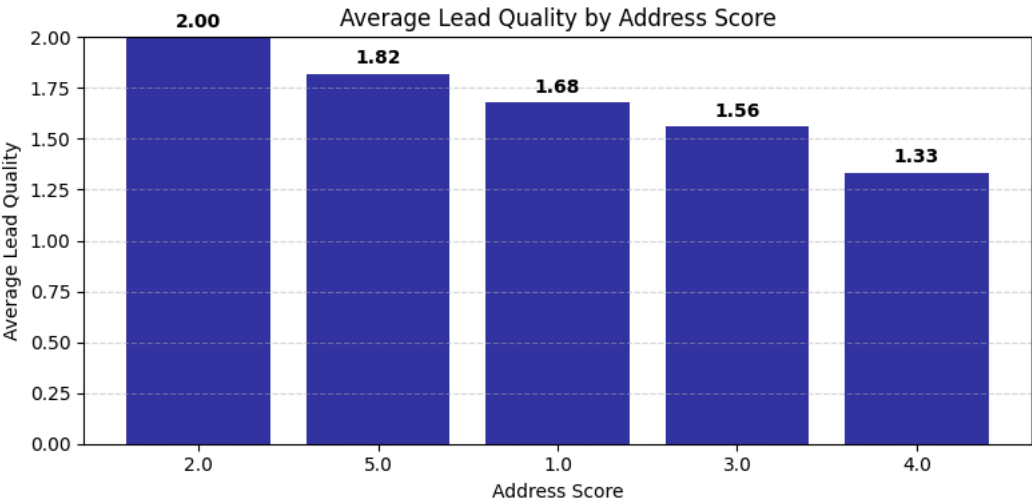


Average Lead Quality by State



```
import matplotlib.pyplot as plt
address_summary = lead_op_clean.groupby('AddressScore').agg(
    avg_quality=('QualityNumeric','mean'),
    lead_count=('QualityNumeric','count')
).sort_values('avg_quality', ascending=False)

plt.figure(figsize=(8,4))
plt.bar(address_summary.index.astype(str), address_summary['avg_quality'], color='darkblue', alpha=0.8)
for i, v in enumerate(address_summary['avg_quality']):
    plt.text(i, v + 0.05, f"{v:.2f}", ha='center', fontweight='bold')
plt.ylabel('Average Lead Quality')
plt.xlabel('Address Score')
plt.title('Average Lead Quality by Address Score')
plt.ylim(0, 2)
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



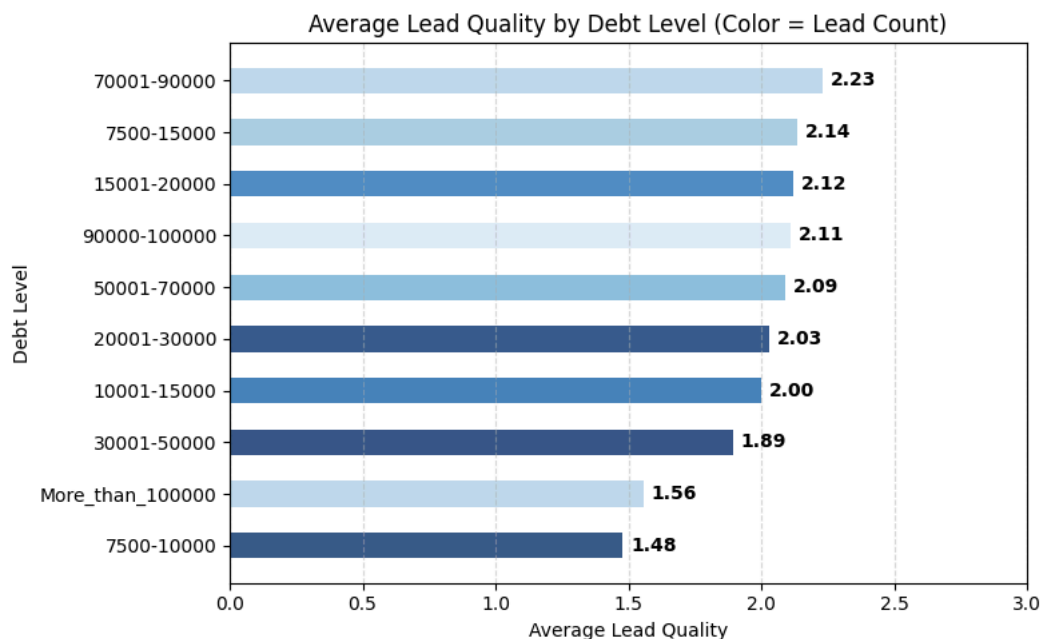
```
lead_op_clean.groupby('DebtLevel').agg(
    avg_quality=('QualityNumeric','mean'),
    lead_count=('QualityNumeric','count')
).sort_values('avg_quality', ascending=False)
```

	avg_quality	lead_count
DebtLevel		
70001-90000	2.228571	35
7500-15000	2.136364	44
15001-20000	2.121951	82
90000-100000	2.111111	18
50001-70000	2.090909	55
20001-30000	2.027523	109
10001-15000	2.000000	88
30001-50000	1.892857	112
More_than_100000	1.555556	36
7500-10000	1.477477	111

```
import pandas as pd
debt_summary = lead_op_clean.groupby('DebtLevel').agg(
    avg_quality=('QualityNumeric', 'mean'),
    lead_count=('QualityNumeric', 'count')
).reset_index()

debt_summary = debt_summary.sort_values('avg_quality', ascending=True)
plt.figure(figsize=(8,5))
bars = plt.barh(debt_summary['DebtLevel'], debt_summary['avg_quality'],
                color=plt.cm.Blues(debt_summary['lead_count']/debt_summary['lead_count'].max()),
                alpha=0.8, height=0.5)
for i, v in enumerate(debt_summary['avg_quality']):
    plt.text(v + 0.03, i, f'{v:.2f}', va='center', fontweight='bold')

plt.xlabel('Average Lead Quality')
plt.ylabel('Debt Level')
plt.title('Average Lead Quality by Debt Level (Color = Lead Count)')
plt.xlim(0,3)
plt.grid(axis='x', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



#what kind of ad did they see?

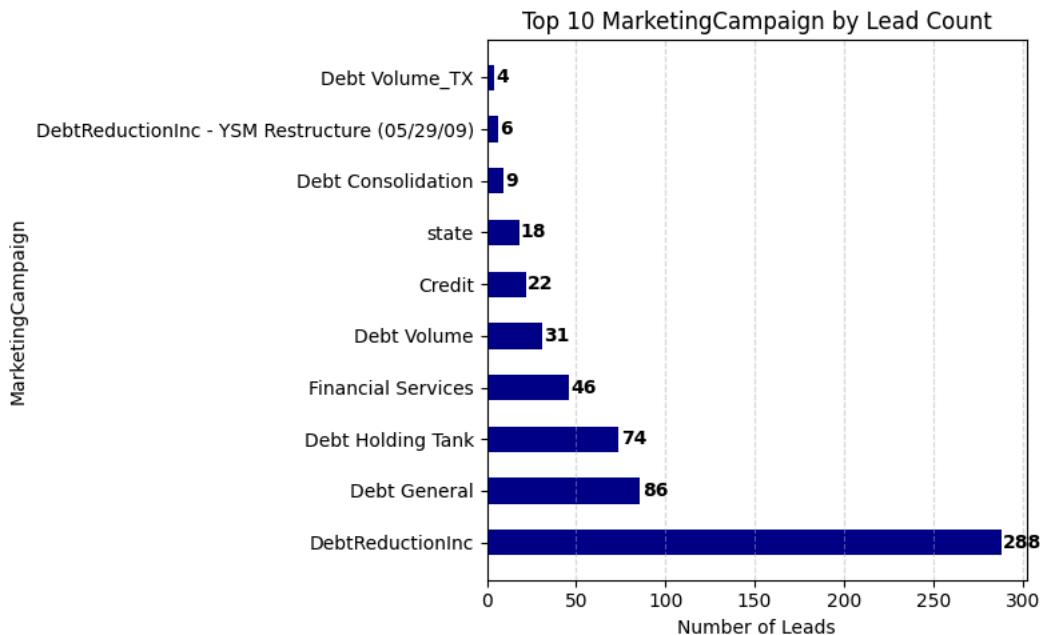
```
campaign_counts = lead_op_clean.groupby('MarketingCampaign').agg(
    lead_count=('QualityNumeric', 'count')
```



```
).sort_values('lead_count', ascending=False)
print(campaign_counts)
```

	lead_count
MarketingCampaign	
DebtReductionInc	288
Debt General	86
Debt Holding Tank	74
Financial Services	46
Debt Volume	31
Credit	22
state	18
Debt Consolidation	9
DebtReductionInc - YSM Restructure (05/29/09)	6
Debt Volume_TX	4
Debt Volume_MI	3
Debt	1
Debt_General_-_SN	1
Debt Volume_CA	1
state - YSM Restructure (05/29/09)	1
Debt_Consolidation_-_SN	1
Credit_-_SN	0
Debt Reduction	0
Debt_Holding_Tank_-_SN	0
Debt_Volume_-_SN	0

```
top_campaigns = campaign_counts.head(10)
plt.figure(figsize=(8,5))
plt.barh(top_campaigns.index, top_campaigns['lead_count'], color='darkblue', height=0.5)
for i, v in enumerate(top_campaigns['lead_count']):
    plt.text(v + 1, i, str(v), va='center', fontweight='bold')
plt.xlabel('Number of Leads')
plt.ylabel('MarketingCampaign')
plt.title('Top 10 MarketingCampaign by Lead Count')
plt.grid(axis='x', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



```
adgroup_counts = lead_op_clean.groupby('AdGroup').agg(
    lead_count=('QualityNumeric', 'count')
).sort_values('lead_count', ascending=False)
print(adgroup_counts)
```

	lead_count
AdGroup	
Holding Tank - Debt	74
Student Debt	71
Consolidate	46
Debt Consolidation	42
Lower Payments	41
...	...
Debt_Services	0



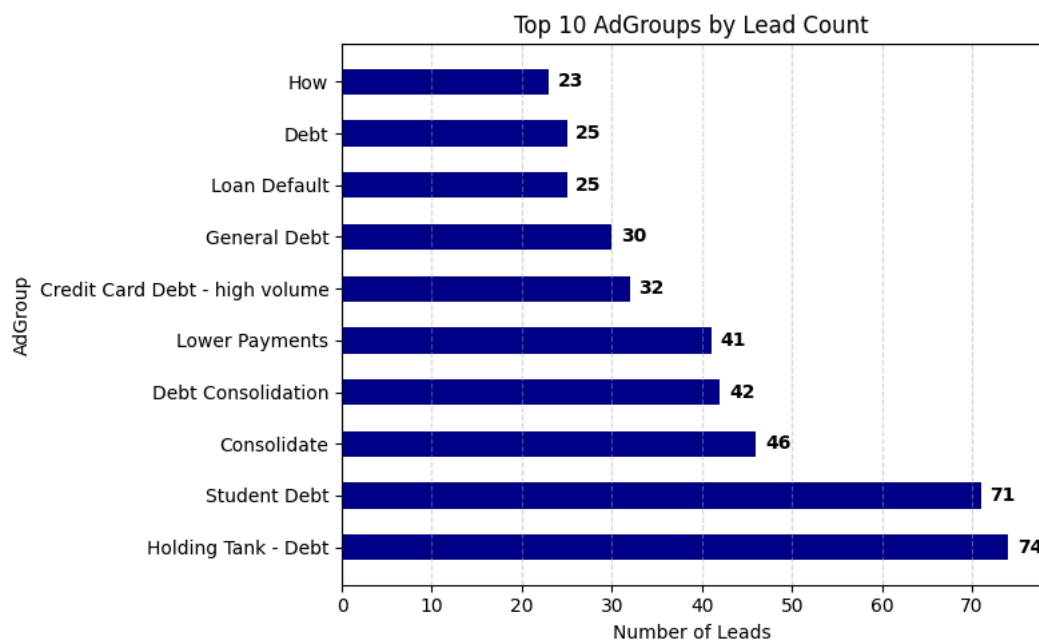
Online Settle	0
Lower Payments (LV)	0
Unsecured Debt	0
Unsecured Debt Consolidation Loan	0

[103 rows x 1 columns]

```
top_adgroups = adgroup_counts.head(10)
```

```
plt.figure(figsize=(8,5))
plt.barh(top_adgroups.index, top_adgroups['lead_count'], color='darkblue', height=0.5)
for i, v in enumerate(top_adgroups['lead_count']):
    plt.text(v + 1, i, str(v), va='center', fontweight='bold')

plt.xlabel('Number of Leads')
plt.ylabel('AdGroup')
plt.title('Top 10 AdGroups by Lead Count')
plt.grid(axis='x', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



Start coding or [generate](#) with AI.

```
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from tabulate import tabulate
lead_op_clean.isnull().sum()
```



	0
WidgetName	0
PublisherZoneName	0
PublisherCampaignName	0
AddressScore	1850
PhoneScore	1628
AdvertiserCampaignName	0
State	0
DebtLevel	0
Partner	0

```
import pandas as pd
import xgboost as xgb
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score

num_cols = ['AddressScore', 'PhoneScore']
for col in num_cols:
    lead_op_clean[col] = lead_op_clean[col].fillna(3)
    lead_op_clean[col] = lead_op_clean[col].astype(int)

debt_mapping = {
    '7500-10000': 1,
    '7500-15000': 2,
    '10001-15000': 3,
    '15001-20000': 4,
    '20001-30000': 5,
    '30001-50000': 6,
    '50001-70000': 7,
    '70001-90000': 8,
    '90000-100000': 9,
    'More_than_100000': 10
}
lead_op_clean['DebtLevel'] = lead_op_clean['DebtLevel'].map(debt_mapping)
lead_op_clean['DebtLevel'] = lead_op_clean['DebtLevel'].fillna(0).astype(int)
cat_cols = [
    'ReferralDomain', 'MarketingCampaign', 'AdGroup', 'Keyword', 'SearchQuery',
    'ReferralURL', 'PublisherZoneName', 'PublisherCampaignName',
    'AdvertiserCampaignName', 'Partner', 'WidgetName', 'LandingPageURL'
]

for col in cat_cols:
    lead_op_clean[col] = lead_op_clean[col].fillna('Unknown')
    le = LabelEncoder()
    lead_op_clean[col] = le.fit_transform(lead_op_clean[col].astype(str))

lead_op_clean = lead_op_clean.dropna(subset=['QualityGroup'])

X = lead_op_clean.drop(columns=['QualityGroup'])
y = lead_op_clean['QualityGroup']

le_y = LabelEncoder()
y_encoded = le_y.fit_transform(y)

print("X shape:", X.shape)
print("y shape:", y_encoded.shape)
```

```
X shape: (690, 18)
y shape: (690,)
```

```
from sklearn.preprocessing import LabelEncoder
for col in X.select_dtypes(include='object').columns:
    le = LabelEncoder()
    X[col] = le.fit_transform(X[col].astype(str))
X_train, X_test, y_train, y_test = train_test_split(
```

